

# "AI does not understand the real world.": AI Augmented Software Development

Eszter Vigh  
Center for Information Security and  
Trust, Theoretical Computer Science  
IT University of Copenhagen  
Copenhagen, Denmark  
esvi@itu.dk

Freja Sunesen  
IT University of Copenhagen  
Copenhagen, Denmark  
fsun@itu.dk

Louise Barkhuus  
IT University of Copenhagen  
Copenhagen, Denmark  
barkhuus@itu.dk

## Abstract

Generative artificial intelligence (AI) tools are increasingly embedded in software development work, reshaping how code is produced, reviewed, and maintained. Following the public release of ChatGPT and the expansion of the AI tooling ecosystem, developers now routinely encounter AI assistance in collaborative and security-sensitive contexts. Through interviews with eleven software developers familiar with generative AI tools and working across teams, we examine how developers adopt and evaluate AI in daily work, how AI use affects security-related practices, and how these tools shape collaboration and accountability. Our findings show that AI use is contingent and selective rather than habitual. Developers primarily rely on AI for low-complexity, low-accountability tasks, while deliberately limiting use in security-critical or system-defining work. Responsibility for correctness and safety is consistently framed as non-transferable leading many developers to refuse or limit AI use.

## CCS Concepts

• **Social and professional topics** → **Computing and business**; • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Programming teams**; **Software development process management**.

## Keywords

Software development, AI tools, teamworking dynamics, cognitive load

### ACM Reference Format:

Eszter Vigh, Freja Sunesen, and Louise Barkhuus. 2026. "AI does not understand the real world.": AI Augmented Software Development. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3799079>

## 1 Introduction

Artificial intelligence (AI) has entered the workforce after an era marred by remote and hybrid working practices, where physical working presence became less significant if work continued to be produced at expected rates [15, 25]. Public media contain endless

discussions on ethics of AI usage, when it should be used, how it should be used, who should have access, and for what tasks AI should be used for. The public release of OpenAI's ChatGPT in November 2022 marked a turning point in this shift and quickly brought generative AI to unprecedented user bases [18]. Since then, the work-AI ecosystem has expanded to include key players such as GitHub Copilot, Google's Gemini, and Anthropic's Claude. These tools promise productivity gains, faster problem solving, and relevant support for both novice and experienced workers particularly software developers. At the same time, they change the workflow and thought process behind how software is written, reviewed, and maintained.

An area of particular concern is secure code development with AI-in-the-loop. While AI tool assistance may accelerate development work, it also introduces new risks, including insecure code suggestions, over-reliance on external (and in some cases foreign) systems, and potential data leakage [28]. Understanding how developers navigate these risks, and how AI tools shape secure coding practices in everyday work is a question that warrants investigation throughout the AI integration process.

Software development is a collaborative work activity [29]. This work is rarely carried out in isolation; instead, developers rely on co-creation in the form of shared codebases, code reviews, and collaborative test case creation to build and maintain systems [26]. The introduction of AI tools has disrupted the collaborative dynamics by influencing when and how developers engage each other, distribute tasks, and negotiate trust and accountability [27, 32]. These developments invite new opportunities and challenges associated with integrating generative AI tools into software development. There are challenges in distinguishing the tools as tools when the amount of work and at times the quality of work is comparable to human input. This interview study focuses on developers working in software development teams through a qualitative approach to investigate how AI tools are approached, evaluated, and integrated in common software development tasks.

This study aims to explore the following research questions:

- (1) **Adoption and Use:** How do software developers approach and integrate generative AI tools in their daily work?
- (2) **Security Practices:** In what ways do these tools affect security-related coding practices?
- (3) **Collaboration and Team Dynamics:** How do AI tools shape collaboration dynamics and knowledge-sharing within teams?



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI EA '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2281-3/26/04  
<https://doi.org/10.1145/3772363.3799079>

By structuring the research questions in this way, the study explicitly connects the technical, organizational, and social dimensions of AI tool adoption in software development teams. This ensures that the analysis moves beyond narrow concerns of productivity to include broader implications for secure and collaborative software development.

## 2 Related Works

### 2.1 AI Impact on Team Dynamics

AI tools are not isolated technical artifacts, but elements embedded in socio-technical systems where meaning, responsibility, and coordination evolve through interaction and shared practice [19]. This evolution informs the analysis of collaboration and interaction practices rather than serving as stand-alone theoretical models. By definition this evolution means that the success, trust, and usability of these systems is different every time they are engaged with. The reliability of these tools changes every model, every agent [24].

Agency and responsibility are central concerns in software development, where decisions about design, implementation, and deployment carry technical, organizational, and societal consequences [1]. In AI-assisted software development, these concerns are complicated by the introduction of generative systems that actively contribute code, suggestions, and explanations, raising questions about intellectual property, control, and accountability. Alami and Ernst's work on accountability and agency in software engineering, conceptualizes responsibility as something enacted and negotiated through concrete practices such as code review, evaluation, and justification of decisions [2, 3]. AI tools do not possess agency in the same sense as human collaborators instead, they reconfigure how human agency is exercised by shifting effort toward evaluation, sense-making, and accountability work.

Previous work on AI-assisted code review shows that developers engage in similar sense-making processes when evaluating feedback from AI tools and from human peers, but that trust, lack of contextual understanding, and absence of reciprocal accountability constrain the adoption of AI-generated suggestions [4]. Accountability mechanisms such as code review, testing, and documentation play a central role in reasserting human control over AI-assisted work by making decisions explainable, contestable, and attributable within teams. Human-AI team collaboration in software development is therefore best understood as a socio-technical practice embedded in collective workflows rather than as a dyadic interaction between an individual and an autonomous system [35]. Collaboration with AI tools requires developers to actively decide when AI-generated artifacts are appropriate, how they should be adapted, and when they should be rejected, with collaborative consensus-building being a pillar in maintaining programming quality standards so that more junior developers continue to gain experience and learn valuable lessons and skills in the work context [36]. These practices are shaped by professional norms, organizational expectations, and shared understandings of acceptable risk.

### 2.2 AI Usage in Small, Medium, and Large Enterprises

Tools such as GitHub Copilot and ChatGPT enable developers to generate, transform, and test code. Prior research has often examined these tools in terms of productivity gains, code quality, and developer experience, frequently reporting efficiency improvements for routine or repetitive programming tasks [6, 10]. These empirical studies of developer interaction with code-generating models show that AI tools are used in different modes depending on task characteristics and developer intent. Barke et al. [6] describe a bimodal pattern of interaction, where developers either use AI tools to accelerate progress toward a known solution or to explore alternative approaches when uncertain. At the same time, a growing body of research highlights important limitations and risks associated with the use of generative AI in software development [11, 12]. Studies have shown that AI-generated code may contain security vulnerabilities, subtle errors, or misleading explanations, particularly in complex or security-critical contexts [20, 21]. As a result, developers are often required to critically evaluate, verify, and contextualize AI outputs rather than treating them as authoritative solutions.

Beyond technical performance, qualitative research emphasizes that the adoption of generative AI tools is shaped by social, organizational, and professional factors [13, 17]. Developers' use of AI tools varies depending on task criticality, perceived risk, and organizational norms, with patterns of selective, situational, and sometimes informal use emerging across contexts [16, 23]. The problem with all of the existing research is it comes out retroactively. By nature of how research is conducted and publication processes, there is no truly accurate representation of the lived experience of developers. There are days where practitioners can count on the AI agent and context window you built, the next day something changes, someone on the team has engaged the agent or modified the context window and the team cannot reproduce the same results as before [37]. The model is only as good as the effort and data that is in the system [31]. This background information is only useful in aggregate form contextualized by the specific use case.

## 3 Methods

This study adopts a qualitative, exploratory research design to investigate how developers in software development teams in small, medium, and large enterprises use, interpret, and negotiate generative AI tools in their everyday work practices. The focus is not on measuring performance outcomes or adoption rates, but on understanding how AI tools are experienced, evaluated, and integrated within professional, organizational, and collaborative contexts. A qualitative approach is particularly suited to this purpose, as it allows for in-depth exploration of participants' subjective experiences, meanings, and professional judgments [9].

To situate these qualitative insights within established research on technology use, this study draws on the Technology Acceptance Model (TAM). TAM has been widely used to explain individuals' adoption of new technologies through the constructs of perceived usefulness (PU) and perceived ease of use (PEOU) [14]. TAM has proven valuable for capturing early-stage adoption decisions and

attitudes toward technology use, particularly in organizational settings. Subsequent extensions of TAM, including TAM2, TAM3, and the Unified Theory of Acceptance and Use of Technology (UTAUT), have sought to incorporate additional factors such as social influence, facilitating conditions, experience, and voluntariness of use [33, 34]. In the context of generative AI tools, developers' engagement is rarely a binary choice of adoption or non-adoption. Instead, use is often selective, situational, and continuously negotiated based on task characteristics, perceived consequences of failure, and professional responsibility [21, 23]. The decision to use AI tools fundamentally shifts the role that the software developers have as AI tools require a different set of skills to effectively incorporate in workflows [5]. The changes to this cognitive load fundamentally impact who and what rate AI is accepted and incorporated into workflows.

Our empirical material consists of interviews with eleven participants who work professionally in software development. Participants were selected using a purposive sampling strategy aimed at recruiting developers with relevant and situated experience of generative AI tools in professional software development contexts. The purpose of this approach was not statistical representativeness, but analytical relevance [9]. The thematic analysis followed the six phases outlined by Braun and Clarke [7, 8]: (1) familiarization with the data, (2) initial coding, (3) searching for themes, (4) reviewing themes, (5) defining and naming themes, and (6) producing the analysis. All interviews were transcribed verbatim and anonymized prior to analysis. Transcripts were reviewed multiple times to ensure familiarity with the material and to support initial sense-making. This process enabled the identification of preliminary patterns, points of interest and recurring topics across interviews and formed the foundation for subsequent coding.

Although presented as distinct phases, the analytical process was recursive rather than linear, involving repeated movement between transcripts, codes and themes. The coding process combined inductive and deductive strategies. Initial codes were generated inductively through close, line-by-line engagement with the interview material, allowing themes to directly correlate to participants' accounts rather than being imposed in advance. Interview excerpts were manually sorted and grouped under evolving thematic codes, enabling systematic comparison across participants and iterative refinement of the coding structure. Codes were used as concise, data-near labels capturing concrete practices, evaluations, and judgments expressed by participants [30]. Through iterative second-cycle coding, related codes were consolidated into higher-level analytical patterns that form the study's themes. Themes are treated as interpretive constructs that articulate shared meanings across the dataset rather than as topical categories.

## 4 Participants

Access to participants was facilitated through a combination of opportunity-based and convenience-oriented recruitment channels. Inclusion criteria required participants to (1) work professionally with software development or closely related technical roles and (2) have at least some familiarity with generative AI tools. For the purposes of this study we did not limit familiarity to using AI tools for work activities, this was to encourage more open discussion

of AI use. In this way there were participants working in software development adjacent teams (e.g. UX/UI teams). The participants each gave informed consent in line with university ethics guidance on human participant consent. The full demographic table of participants can be found in Table 1.

## 5 Findings

Participants described the adoption of generative AI tools as a contingent and situational practice rather than a stable preference. Decisions to use or avoid AI were shaped by how useful the AI tool was for a specific task, the effort required to get a 'useful' response, organizational support, social norms, and risk. AI tools were widely used for low-complexity and low-accountability tasks such as summarization, translation, drafting, and boilerplate generation. For example, P11 described how AI supported routine documentation work during organizational growth: *"We had a lot of documentation on another platform written in [another language]. We copied the text into our company ChatGPT and asked it to translate to English and convert to Markdown. Then I could put it into the code in a new repository and use that as a starting point. It's a very good way to move documentation from one format to another."* In contrast, participants consistently reported that AI use declined sharply for tasks involving system correctness, architectural coherence, or long-term maintainability. As P9 explained, AI tools were least useful *"[for] anything requiring deep architectural understanding. It does not have the context. Sometimes it confidently invents things that make no sense."* Developers selectively utilized AI use based on task criticality and the perceived reliability of AI outputs. Developer time and effort remained valuable, and there was little value to arguing with AI tools when developers themselves were more efficient. This practice of boundary setting with AI by way identifying the optimal use-cases is just one way that workflows and tasks are shifting within software development teams.

Security and accountability concerns shaped AI "appropriate" task boundaries. Across participants, responsibility for correctness, safety, and maintainability was framed as non-transferable, even when AI-generated artifacts were involved. As a result, AI tools were deliberately excluded from security-critical workflows, particularly those involving authentication, authorization, or data exposure. P7 emphasized the role of consequences in these decisions: *"If AI makes a mistake in a layout, then it's annoying, but to live with. If AI makes a mistake in something that affects rights, authentication or exposure of data, it's a completely different league. I want to make sure that I understand what is going on line by line, otherwise I cannot vouch for it."* When AI-generated code was used in higher-stakes contexts, participants described reclaiming control through line-by-line review, testing, and extensive rewriting. P4 explicitly rejected the idea that responsibility could be deferred to the model: *"I would not feel comfortable saying 'AI wrote this, so it is fine.' It would feel like a cop out to blame the AI if something went wrong. In the end someone, or some team, will be held responsible, and I think that should be us, not 'the model'."* Trust in AI was described as fragile and easily undermined by hallucinations or subtle errors, leading developers to reduce reliance over time and treat AI as a fallible assistant rather than an authoritative source. P6 explained it as: *"I do think about it, but I am not in panic mode. I do not mind*

**Table 1: Participant demographics and AI tool usage**

| P#  | Age   | Gen. | Exp. | Work domain(s)               | Enterprise Size | Language(s)               | AI freq. | Work Use | Work AI tools                                          | Personal Use | Personal AI tools         |
|-----|-------|------|------|------------------------------|-----------------|---------------------------|----------|----------|--------------------------------------------------------|--------------|---------------------------|
| P1  | 35–44 | M    | 10+  | System integration, web dev. | S               | Scala                     | Rarely   | Yes      | ChatGPT                                                | No           | N/A                       |
| P2  | 25–34 | M    | 5–9  | System integration, web dev. | S               | Scala                     | Never    | No       | N/A                                                    | No           | N/A                       |
| P3  | 25–34 | M    | 1–4  | System integration, web dev. | S               | Python, Scala             | Never    | No       | N/A                                                    | No           | N/A                       |
| P4  | 25–34 | F    | 1–4  | Web/app dev., full-stack     | L               | C#, Java, JS, Python, SQL | Daily    | Yes      | Internal AI, ChatGPT, Gemini                           | Yes          | ChatGPT, Gemini, DeepSeek |
| P5  | 45–54 | F    | 20+  | UX/UI, HCI                   | L               | Figma                     | Daily    | Yes      | ChatGPT, MS Copilot, Loveable, Protopie                | Yes          | ChatGPT                   |
| P6  | 25–34 | M    | 1–4  | Full-stack web dev.          | S               | JS, Node, React, SQL      | Weekly   | Yes      | GitHub Copilot, ChatGPT                                | No           | N/A                       |
| P7  | 25–34 | M    | 1–4  | Front-end web dev.           | S               | JS                        | Daily    | Yes      | ChatGPT, MS Copilot, GitHub Copilot                    | Yes          | ChatGPT, Gemini           |
| P8  | 25–34 | M    | 10+  | Backend, DevOps              | S               | JS, Node, PHP, Python     | Daily    | Yes      | GitHub Copilot, ChatGPT                                | Yes          | Not Disclosed             |
| P9  | 45–54 | M    | 5–9  | Front-end web dev.           | S               | JS                        | Daily    | Yes      | Warp, Gemini, ChatGPT                                  | Yes          | Not Disclosed             |
| P10 | 25–34 | M    | 1–4  | Backend dev. (.NET, API)     | M               | .NET, C#, SQL             | Daily    | Yes      | ChatGPT, GitHub Copilot, Internal AI System, Grammarly | Yes          | ChatGPT                   |
| P11 | 55+   | F    | 20+  | Backend dev. (.NET, API)     | M               | .NET, C#, SQL             | Daily    | Yes      | ChatGPT, GitHub Copilot                                | Yes          | ChatGPT                   |

Enterprise size: Small (<50 employees), Medium (50–500), Large (>500).

*that AI exists, and I do not mind using it. If we reach a point where there is 'no need to learn anymore', that would worry me, but I do not think we will get exactly there. I think AI will take over more repetitive parts and assist more, but not remove the need for humans to reason about systems, constraints and ethics. Because of things like hallucinations, there will always be a need for someone to sanity check what is produced."*

Beyond task execution, participants reflected on how AI tools affect learning, creativity, and collaboration within teams. Several expressed concern that heavy reliance on AI could erode developers' problem-solving skills. P1 described feeling that AI use weakened their own abilities: *"After a weekend where I had it write a lot of code, I could feel that my own creative sense and my 'muscles' for writing code were more rusty."* P1 discontinued LLM usage following the experience. This is in-line with existing findings which found that there are long-term educational consequences of LLM usage [22]. This concern was echoed by P10, who warned that *"It can be dangerous if you lose the learning part. Part of solving a task is to become smarter, and it makes you a better developer."* At the same time, participants reported AI tools did not replace existing practices of collaboration or knowledge-sharing. Developers working with legacy or highly customized systems emphasized the continued importance of human expertise. P3, like P1, did not use AI in their workflow. As P3 explained, *"I'm working on a legacy codebase... so often I start by sparring with colleagues who have been here longer and know the codebase better."* Organizational constraints further shaped these dynamics: P5 described strict workplace policies that limited AI use due to security concerns, *"At work we are locked down. We are not allowed to use open AI browsing because of security concerns. If we try to use tools like ChatGPT or Copilot in a browser, we get a pop up warning and are reminded not to share any sensitive data."* Tool integration and organizational endorsement influenced adoption choices, with participants favoring AI tools that were officially supported and embedded in existing workflows. As P8 noted, *"The model matters, but integration and support matter a lot too."* Participants positioned AI tools as supplementary aids that offload routine cognitive labor while reinforcing the centrality of human judgment, shared accountability, and team-based validation. Participants positioned AI tools as supplementary aids that offload routine cognitive labor while reinforcing the centrality of human

judgment, shared accountability, and team-based validation. In several instances people did not use AI tools as a conscious choice to uphold these dynamics. This overall supports the emerging practice of AI boundary setting in software development teams.

## 6 Conclusion

Generative AI tool adoption in professional software development cannot be understood as a binary of acceptance or resistance. Instead, our findings show that AI use emerges as a situated, risk-bounded, and organizationally embedded practice in which developers continuously negotiate usefulness, responsibility, and trust. Consistent with TAM [33, 34], developers do not simply "adopt" AI tools; they actively calibrate their engagement based on task criticality, perceived consequences of failure, organizational policy, how their work-tasks already engage their team, and prior experiences with AI errors. These practices reflect early concepts from TAM where perceived usefulness as a broad notion, was part of the trajectory of acceptance [14]. In several cases people had deliberately decided not to use AI tools, either from a bad experience or from the need to feel they were in control of their code. This ongoing negotiation results in fluid and frequently reconfigured workflows, where AI is selectively invoked, constrained, or deliberately excluded. This is particularly relevant in the context of security-related tasks where some companies had moved to block access to AI tools altogether and reminding employees of the dangers of inputting sensitive data into the tools. The adoption of AI tools introduces new forms of cognitive and organizational labor. Rather than reducing effort, AI use often shifts work toward oversight, validation, and contextualization. Developers described the need to constantly assess when AI is appropriate, interpret and verify its outputs, and reassert control in higher-stakes scenarios. The cognitive load is not less, it is just different. Developers remain the final arbiters of what enters the codebase and who is accountable when systems fail. In practice, the role of the software developer is evolving to encompass new forms of work, including compliance with organizational AI policies, active management of what contextual information is shared with AI systems, and the orchestration of AI outputs within complex socio-technical constraints.

## Acknowledgments

This work was funded by Digital Research Centre Denmark under the project "Understanding and Supporting Secure AI Tools in Software Development Teams".

## References

- [1] Daniel Ajiga, Patrick Azuka Okeleke, Samuel Olaoluwa Folorunsho, and Chinedu Ezeigweneme. 2024. Navigating ethical considerations in software development and deployment in technological giants. *International Journal of Ethics and Research Utilization* 7, 1 (2024).
- [2] Adam Alami and Neil Ernst. 2024. Understanding the building blocks of accountability in software engineering. In *Proceedings of the 2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering*. 153–163.
- [3] Adam Alami and Neil Ernst. 2025. Human and Machine: How Software Engineers Perceive and Engage with AI-Assisted Code Reviews Compared to Their Peers. In *2025 IEEE/ACM 18th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE)*. IEEE, 63–74.
- [4] Adam Alami, Victor Jensen, and Neil Ernst. 2025. Accountability in code review: The role of intrinsic drivers and the impact of llms. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 1–44.
- [5] Mamdouh Alenezi and Mohammed Akour. 2025. Ai-driven innovations in software engineering: a review of current practices and future directions. *Applied Sciences* 15, 3 (2025), 1344.
- [6] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [7] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [8] Virginia Braun and Victoria Clarke. 2021. Thematic analysis: A practical guide. (2021).
- [9] Alan Bryman. 2016. *Social research methods*. Oxford university press.
- [10] Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. 2025. Generative AI at work. *The Quarterly Journal of Economics* 140, 2 (2025), 889–942.
- [11] Kevin Buehler, Rachel Dooley, Liz Grennan, and Alex Singla. 2021. Getting to know—and manage—your biggest AI risks. *McKinsey and Company* (2021).
- [12] Benjamin Cheatham, Kia Javanmardian, and Hamid Samandari. 2019. Confronting the risks of artificial intelligence. *McKinsey Quarterly* 2, 38 (2019), 1–9.
- [13] Sarah J Daly, Anna Wiewiora, and Greg Hearn. 2025. Shifting attitudes and trust in AI: Influences on organizational AI adoption. *Technological Forecasting and Social Change* 215 (2025), 124108.
- [14] Fred D Davis. 1989. Perceived Usefulness, Perceived Ease of Use and User Acceptance of Information Technology. *MIS quarterly* (1989).
- [15] Melanie Duckert, Louise Barkhuus, and Pernille Bjørn. 2023. Collocated distance: a fundamental challenge for the design of hybrid work technologies. In *Proceedings of the 2023 CHI conference on human factors in computing systems*. 1–16.
- [16] Lorentsa Gkinko and Amany Elbanna. 2023. The appropriation of conversational AI in the workplace: A taxonomy of AI chatbot users. *International Journal of Information Management* 69 (2023), 102568.
- [17] Ella Glikson and Anita Williams Woolley. 2020. Human trust in artificial intelligence: Review of empirical research. *Academy of management annals* 14, 2 (2020), 627–660.
- [18] Abid Haleem, Mohd Javaid, and Ravi Pratap Singh. 2022. An era of ChatGPT as a significant futuristic support tool: A study on features, abilities, and challenges. *Benchmark transactions on benchmarks, standards and evaluations* 2, 4 (2022), 100089.
- [19] Andrew JI Jones, Alexander Artikis, and Jeremy Pitt. 2013. The design of intelligent socio-technical systems. *Artificial Intelligence Review* 39, 1 (2013), 5–20.
- [20] Raphaël Khoury, Anderson R Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How secure is code generated by chatgpt?. In *2023 IEEE international conference on systems, man, and cybernetics (SMC)*. IEEE, 2445–2451.
- [21] Jan H Klemmer, Stefan Albert Horstmann, Nikhil Patnaik, Cordelia Ludden, Cordell Burton Jr, Carson Powers, Fabio Massacci, Akond Rahman, Daniel Votipka, Heather Richter Lipford, et al. 2024. Using ai assistants in software development: A qualitative study on security practices and concerns. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2726–2740.
- [22] Nataliya Kosmyna, Eugene Hauptmann, Ye Tong Yuan, Jessica Situ, Xian-Hao Liao, Ashly Vivian Beresnitzky, Iris Braunstein, and Pattie Maes. 2025. Your brain on ChatGPT: Accumulation of cognitive debt when using an AI assistant for essay writing task. *arXiv preprint arXiv:2506.08872* 4 (2025).
- [23] Jiachen Li, Elizabeth D Mynatt, Varun Mishra, and Jonathan Bell. 2025. 'Always Nice and Confident, Sometimes Wrong': Developer's Experiences Engaging Generative AI Chatbots Versus Human-Powered Q&A Platforms. *Proceedings of the ACM on Human-Computer Interaction* 9, 2 (2025), 1–22.
- [24] Thomas Mee, Justin J Choi, and Amiran Baduashvili. 2025. Same Prompt, Different Answer: Why AI Disagrees with Itself. 3 pages.
- [25] Milad Mirbabaie, Felix Brünker, Nicholas RJ Möllmann Frick, and Stefan Stieglitz. 2021. The rise of artificial intelligence—understanding the AI identity threat at the workplace. *Electronic Markets* 32, 1 (2021), 73–99.
- [26] Inah Omoronyia, John Ferguson, Marc Roper, and Murray Wood. 2009. Using developer activity data to enhance awareness during collaborative software development. *Computer Supported Cooperative Work (CSCW)* 18, 5 (2009), 509.
- [27] Veronica Pimenova, Sarah Fakhoury, Christian Bird, Margaret-Anne Storey, and Madeline Endres. 2025. Good Vibrations? A Qualitative Study of Co-Creation, Communication, Flow, and Trust in Vibe Coding. *arXiv preprint arXiv:2509.12491* (2025).
- [28] Petar Radanliev, David De Roure, Carsten Maple, Jason RC Nurse, Razvan Nicolescu, and Uchenna Ani. 2024. AI security and cyber risk in IoT systems. *Frontiers in Big Data* 7 (2024), 1402745.
- [29] Pierre N Robillard and Martin P Robillard. 2000. Types of collaborative work in software engineering. *Journal of Systems and Software* 53, 3 (2000), 219–224.
- [30] Johnny Saldaña. 2021. The coding manual for qualitative researchers. (2021).
- [31] Nithya Sambasivan, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M Aroyo. 2021. "Everyone wants to do the model work, not the data work": Data Cascades in High-Stakes AI. In *proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–15.
- [32] Vijayanth S Tummala, Tashieka S Burris-Melville, and Thomas C Eskridge. 2025. AI as a Team Member: Redefining Collaboration. *Journal of Leadership Studies* 18, 4 (2025), 67–80.
- [33] Viswanath Venkatesh and Hillol Bala. 2008. Technology acceptance model 3 and a research agenda on interventions. *Decision sciences* 39, 2 (2008), 273–315.
- [34] Viswanath Venkatesh, Michael G Morris, Gordon B Davis, and Fred D Davis. 2003. User acceptance of information technology: Toward a unified view1. *MIS quarterly* 27, 3 (2003), 425–478.
- [35] Toni Waeffer and Ute Schmid. 2020. Explainability is not enough: requirements for human-AI-partnership in complex socio-technical systems. (2020).
- [36] Cai-Ling Wang and Yuan-Chi Tseng. 2025. Enhancing consensus-building in collaborative design: a systematic review of digital design tools. In *International Conference on Human-Computer Interaction*. Springer, 254–272.
- [37] Katharina A Zweig. 2022. *Awkward intelligence: Where AI goes wrong, why it matters, and what we can do about it*. MIT Press.